

# 028 练习与实验详解

## 1 两种一阶与二阶路线

展开  $f = (x^4 + 2x^3 + x^2)/2$ , 得  $f' = 2x^3 + 3x^2 + x$ , 再得  $f'' = 6x^2 + 6x + 1$ 。保留复合结构则令  $r=x^2+x$ , 先得  $f' = r(2x + 1)$ , 再用乘法法则得  $f'' = (2x + 1)^2 + 2r$ , 与展开式一致。

在  $x=1/2$  处,  $r=3/4$ ,  $f=9/32$ ,  $f'=3/2$ ,  $f''=11/2$ 。两条符号路线使用不同表达方式, 不代表程序已经构造了可再次自动求导的梯度图。

## 2 手算JVP与VJP

$J = \begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix}$ , shape为(2,2), 行F1/F2、列x1/x2。  $v=(3,-1)$ 和 $t=(1/2,2)$ 均为二维列向量。

$$J^T v = (-1 \cdot 3 + 1 \cdot (-1), 2 \cdot 3 + (-2)(-1))^T = (-4, 8)^T.$$

$$Jt = (-1/2 + 4, 1/2 - 4)^T = (3.5, -3.5)^T.$$

左内积为  $3 \times 3.5 + (-1) \times (-3.5) = 14$ ; 右内积为  $(-4) \times 0.5 + 8 \times 2 = 14$ 。返回VJP与输入同shape, JVP与输出同shape。这里维度恰好相同不能掩盖方向区别。

## 3 全1种子

全1种子得到  $\nabla_x \sum_i F_i(x) = J^T \mathbf{1}$ , 不是完整J。若要逐行恢复J, 可以分别给每个输出基向量种子, 做多次VJP, 但这会增加时间和结果存储。

对手算例, J两行相加为(0,0)。这说明F1+F2在该点一阶变化为0, 并不说明每一个F输出都局部不变。相加时方向抵消了。

## 4 第二行种子置零

C改为[[1,2,3],[0,0,0]]后, u梯度为[[6],[0]], v梯度为[[1,2,3]]。第二行输出不参与当前加权目标, 但第一行仍使用所有v坐标。不能把整个共享参数v的梯度也清零。

## 5 多轴广播

A形状(2,1,3), B形状(1,4,1), 共同输出(2,4,3)。A的axis1由1扩成4, 故对上游沿axis1相加并keepdims=True, 得到(2,1,3)。B的axis0和2分别由1扩成2和3, 沿这两轴相加并保留它们, 得到(1,4,1)。

若B原来只有shape(4,1)，则先处理输出多出来的前导轴，再处理最后一个扩展轴，最终必须回到(4,1)，不是(1,4,1)。

## 6 reshape不是归约

C中有六个独立上游坐标，而u只有两个原坐标。u第一项被三个输出使用，必须把1+2+3合成6；第二项必须把4+5+6合成15。reshape只改变元素排列视图或布局，不会把六个数变成两份和，也不能一般地改变元素个数。

## 7 sum的轴恢复

keepdims=True保留长度1的归约轴，上游可直接广播回输入。keepdims=False删除这些轴，反向要在原轴位置重新插入长度1，再复制到输入各坐标。例如X为(2,3,4)，对axis=(0,2)求和后输出(3,)；种子必须先变成(1,3,1)，再扩到(2,3,4)。

不能把(3,)直接按右对齐规则广播到(2,3,4)，因为3会错误地尝试对齐最后的4。轴的位置和轴的大小同样重要。

## 8 H被使用三次

$H^*H$ 有两个输入位置，加上最后的+H一条，合计三份局部贡献。设Z的上游为G，则回H为 $G^*H+G^*H+G=G^*(2H+1)$ 。节点H的反向规则执行一次，但它的输入梯度在此之前要收齐这三份贡献。

## 9 一个精确梯度坐标

$A[0,1]=0.5 \times (-0.25)+0.125=0$ ， $H[0,1]=\tanh(0)=0$ 。均值目标在该位置的 $Q=(2 \times 0+1)(1-0^2)/6=1/6$ 。对 $X[0,1]$ 还要乘 $w[1]=-1/4$ ，所以梯度为 $-1/24 \approx -0.0416666667$ 。

这一个坐标恰好有简单值，适合手算；其他坐标涉及tanh非零点，应与高精度参照或容差检查对齐，而不是强求十进制字符串完全等于纸上近似。

## 10 seed改变的是目标

默认seed每项1/6，把所有Z坐标作均值。非均匀种子包含负数，目标变成不同位置的加权和；b被所有位置共享，因此它收到的正负贡献也改变。默认 $gb \approx 0.8183867$ ，加权 $gb \approx -0.2084006$ 并不矛盾。

前向参数与数值不变，模型没有因此“变坏”或“变好”。如果seed并非你想研究的目标对应的上游梯度，那么引擎可以完全正确地算出一个错误问题的答案。

## 11 非光滑组合反例

$x > 0$ 时,  $\text{ReLU}(x)=x$ 、 $\text{ReLU}(-x)=0$ ;  $x < 0$ 时, 前者0、后者 $-x$ ;  $x=0$ 时两者都0。所以整体恒等于 $x$ , 普通导数处处1。

但在0处, 两个局部ReLU按引擎约定都给0。第一条贡献0, 第二条的负号与内部 $-x$ 还各乘-1, 但仍有局部0因子, 因此总和0。这是局部约定在复合图中的结果, 并不是普通导数的数值逼近误差。选择不同代数表达式可能改变零点反向行为, 应在模型设计时明确。

## 12 梯度屏障不是新的普通微分定理

正常 $x^3$ 在 $x=2$ 处值8、导数 $3x^2=12$ 。屏障把 $a=x^2$ 当前值4保存为独立叶子, 阻止对原 $x$ 追踪 $a$ 路径。剩余目标相对于原 $x$ 局部看作 $4 \times x$ , 所以返回4。

程序对每个新 $x$ 重跑时前向仍数值等于 $x^3$ , 因此普通数值函数的导数仍为12。屏障的4属于显式指定的算法导数规则, 不应在论文中不加解释地称为 $x^3$ 的精确导数。意外调用detach和主动设计梯度屏障的意图不同, 数值现象却可能相同。

## 13 5.5如何得到?

先人工推导 $df=(x^2+x)(2x+1)$ , 然后用Tensor乘法与加法重新构建这条表达式。vjp对这张新图仍只做一次一阶反向, 得到 $df$ 对 $x$ 的导数5.5。它没有从第一次返回的普通NumPy数组1.5中自动恢复 $x$ 依赖。

保留前向图以便再次求同一个一阶导数, 与创建可继续求导的导数图, 功能不同。此引擎仅实现前者的只读重复使用; 不实现自动高阶图。

## 14 三种累计的区别

一张图内, 多条路径共同决定同一个目标的导数, 必须相加。重复vjp调用则是两个独立请求, 本实现每次初始化新字典, 结果不自动相加。跨微批训练若要累计, 使用者需要显式组合各次结果, 并根据批次大小与损失归约定义权重。

这些是不同层次。仅说“梯度要清零”不够, 需要说明谁持有哪个累计容器、何时加入、何时重置, 才能检查实现是否对应预定目标。

## 15 差分通过后的限制

有限差分检查可以支持“这里的局部数值变化与给定导数规则一致”, 但不证明损失反映真实任务价值、不证明数据没有泄漏、不证明训练收敛到所需解、不证明泛化或因果效应, 也不检查数据使用授权。

此外, 它只覆盖被测点、步长、类型、形状与运算。自动微分的数学成立条件还包括光滑性; 屏障、自定义导数和不可微节点约定需要单独解释。

## 实验A 向量输出

给定代码打印[-2,3]、-4和8。改用[1,1]时两个输入梯度均0，因为目标 $F_1+F_2$ 在此点的一阶变化抵消。  
`vjp(out)`必须拒绝，因为`out`不是标量；自动假设全1种子会悄悄选择“输出求和”这个额外目标。

基于常数掩码的写法只是把两个标量结果放进不同输出位置；没有给引擎新增拼接API。掩码本身会作为常数叶子存在，使用者不应把其形式敏感度误当待训练参数。

## 实验B 乘法广播的手算

$Z=u*v$ 时， $u$ 第一行收到 $1\times 0.5+2\times (-1)+3\times 1.5=3$ ；第二行收到 $4\times 0.5+5\times (-1)+6\times 1.5=6$ 。因此  
 $gu=[[3],[6]]$ 。

$v$ 各列收到两行贡献： $1\times 1+4\times 2=9$ ， $2\times 1+5\times 2=12$ ， $3\times 1+6\times 2=15$ ，所以 $gv=[[9,12,15]]$ 。先乘局部  
 另一个输入，再按来源相加，顺序不能替换成先归约后随意乘一个值。

## 实验C 文件账本

`coordinates.csv`有29个节点坐标行： $X_6$ 、 $w_3$ 、 $b_1$ 、 $a_6$ 、 $h_6$ 、 $z_6$ 、 $L_1$ 。`vjp.csv`有 $X/w/b$ 共10行；  
`finite_difference.csv`有9个步长 $\times 10$ 坐标=90行；`boundaries.csv`有6个边界案例；`summary`保存整体  
 梯度、广播与小型雅可比手算。

默认图有11个节点、12条输入边。均值与加权梯度和普通Python标量参照的最大差分别约 $2.78e-17$ 与  
 $2.22e-16$ ，属于本次浮点路径的末位差异，不是所有环境上的误差上界。

## 实验D 预期现象

D1返回0与1的差异来自局部不可微约定组合；D2的12与4来自显式阻断依赖；D3的1.5与5.5分别是一  
 阶数值与人工重建导数表达式后的一阶结果。三种情况的原因不同，不能都归因于“差分不准”。

改动Tensor创建前的原始NumPy输入之后，既有图仍保留创建时的数值。修改`value`取出的防御性副  
 本也不会改图。参数改变后需新建Tensor并重新执行前向。

## 实验E 输出保护与解释

坏配置、错种子、错归约轴和数值失败应明确报错，不产生貌似成功的新结果。脚本完成全部计算和序  
 列化后才创建输出目录，失败测试要求新目录不存在、旧文件字节不变。

这个保护不包括磁盘中途损坏、进程崩溃或恶意并发替换路径的事务保证。原始配置与结果摘要守卫属  
 于固定教学图文的一致性检查，不是现实数据有效性的证明。